

Iterators & push_back Complexity

C++ Հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը ընդգրկում է STL iterator-ների բոլոր TOP interview թեմաները՝ ինչ է iterator-ը, `begin()` / `end()`, iterator-ների 5 տեսակները, iterator invalidation, erase idiom, reverse/const iterator, `++it` vs `it++`, `advance` / `next` / `prev`, plus `push_back` -ի amortized $O(1)$ complexity-ն մանրամասն: Ամեն մեկը կոդով և interview պատասխանով:

1. Ի՞նչ է Iterator-ը

Iterator-ը object է, որ թույլ է տալիս շարժվել container-ի էլեմենտների մեջ. կարելի է patkeracnel vorpes «xelaci pointer».

```
vector<int> v = {10, 20, 30, 40};  
auto it = v.begin(); // գնւյգ է talis araji element-in  
cout << *it;         // 10 (dereference)  
++it;                // sharzum araj  
cout << *it;         // 20
```

begin() և end()

```
v.begin() // araji element-i iterator  
v.end()   // VERJINIC HETO gtnvox diry (nչ verjin element-y!)
```

```
10 20 30 40  
 ^      ^  
begin()  end() (verjinic heto)
```

Iteration

```
for (auto it = v.begin(); it != v.end(); ++it)  
    cout << *it << " "; // 10 20 30 40
```

Interview trap — end()-y verjin element-n e?

```
auto it = v.end();  
cout << *it; // UNDEFINED BEHAVIOR -- end()-y verjinic heto e, nչ element
```

2. Ինչնի՞ Iterator (ete index ka)

Vector-um ka index (`v[0]`), bayc STL-y generic e. `list`, `set`, `map` **index chunen**. Iterator-y bolor container-neri hamar **miasnakan interface** e talis.

```
// nuyn kody ashxatum e tarber container-neri hamar
template<class Container>
void printAll(const Container& c) {
    for (auto it = c.begin(); it != c.end(); ++it)
        cout << *it << " ";
}
```

3. Iterator-ի 5 տեսակներ

1. Input Iterator	-> միայն կարձակ (++, մեկ անգամ)	
2. Output Iterator	-> միայն գրել	
3. Forward Iterator	-> ++ (միայն առաջ)	forward_list
4. Bidirectional Iterator	-> ++ --	list, set, map
5. Random Access Iterator	-> it+5, it-2, it[3]	vector, deque

Random Access — vector/deque

```
vector<int> v = {1,2,3,4,5};
auto it = v.begin();
it + 3;           // OK -> քաշ 3 ճիշտ առաջ
it[2];           // OK
```

Bidirectional — list/set/map

```
list<int> lst = {1,2,3,4,5};
auto it = lst.begin();
++it; --it;      // OK
// it + 3;       // COMPILE ERROR -- list-ը ռանդոմ ասես չԻՄԻ
```

Interview — ինչու list-ում `it + 5` չի աշխատում

Գրի՛ր որ `vector`-ը ունի Random Access Iterator (սխալ քաշ), իսկ `list`-ը՝ Bidirectional Iterator (միայն ++/--). list-ի նոդաները ցրված են, սխալ հասցե-ներ քաշ հնարավոր չէ.

4. advance / next / prev

List-i (kam ayl non-random-access) iterator-y tegapoxelu hamar.

```
list<int> lst = {10,20,30,40,50};

auto it = lst.begin();
advance(it, 3);           // it -> 3 qayl araj (40), voidn e (poxum e it-y)

auto it2 = next(lst.begin(), 3); // nor iterator, 3 qayl araj -> 40
auto it3 = prev(lst.end(), 1);   // 1 qayl het -> 50
```

- `advance(it, n)` — poxum e IREN it-y (return voch mi ban)
- `next(it, n)` — veradarcnum NOR iterator (araj)
- `prev(it, n)` — veradarcnum NOR iterator (het)

Random access iterator-i (vector) hamar `it + n` avelin arag e; `advance` -y ashxatum e bolor tesaknerum.

5. const_iterator

```
vector<int> v = {1,2,3};  
vector<int>::const_iterator it = v.begin();    // kam auto it = v.cbegin();  
  
cout << *it;    // OK, kardal  
// *it = 10;    // COMPILE ERROR -- const_iterator-ov chi kareli poxel
```

`cbegin()` / `cend()` talis en `const_iterator`. Ogtagorci, erb miayn kardum es (safety + intent):

6. Reverse Iterator

```
vector<int> v = {1,2,3,4};  
for (auto it = v.rbegin(); it != v.rend(); ++it)  
    cout << *it << " ";    // 4 3 2 1 (het karg)
```

`rbegin()` -> verjin element; `rend()` -> araji element-ic araj. `++it` -y reverse iterator-i hamar sharzum e HET.

7. Iterator Invalidation (TOP harc)

Vector — reallocation-i zham invalid

```
vector<int> v = {1,2,3};
auto it = v.begin();
v.push_back(4);      // ete reallocation -> it INVALID (dangling)
cout << *it;        // UNDEFINED BEHAVIOR
```

Vector-um invalid en anum. `push_back`, `insert`, `erase`, `resize`, `reserve` (reallocation-i depqum):

List — stable

```
list<int> lst = {1,2,3};
auto it = lst.begin();
lst.push_back(4);    // it MNUM e VALID
cout << *it;        // OK
```

List-um insert/push chi invalidate anum (miayn erase-y` jnjvac node-iny): Sa list-i mec aʔavelutyunneric e.

8. erase() ʌ iterator (sirvac harc)

SXAL — erase heto it-y invalid e

```
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 5)
        v.erase(it);           // VTANGAVOR -- it-y erase-ic heto invalid, ++it -> UB
}
```

CHISHT — erase veradarcnum e hajord iterator

```
for (auto it = v.begin(); it != v.end(); ) {
    if (*it == 5)
        it = v.erase(it);      // erase veradarcnum e hajord valid iterator
    else
        ++it;
}
```

Kanon. `erase()` -y veradarcnum e jnjvac **hajord** valid iterator-y; ogtagorci ayn, u `++it` mi ara erase-i chuxtin.

9. pointer vs iterator

```
int* p; // miayn memory address
vector<int>::iterator it; // container-aware object
```

- Pointer` unix memory hasce.
 - Iterator container-i «hascee» abstraction; karox e ners pointer chlinel (list-i iterator-y node-i wrapper e). Nuynisk vector`-i iterator-y hachax pointer e, bayc erashxavorvac chł.
-

10. ++it vs it++ (sireli harc)

```
++it;    // pre-increment: poxum, veradarcnum nor vichak (reference)
it++;    // post-increment: pahum hin vichak, poxum, veradarcnum HIN (copy)
```

Iterator-neri hamar **naxentreli e** `++it` (pre-increment), qani or `it++` -y karox e steghcel zhamanakavor (temporary) object -> mi qich tañk, hatkapes canr iterator-neri hamar.

11. push_back() Complexity — Amortized O(1)

Normal depq — O(1)

```
vector<int> v;  
v.push_back(10); // tex ka capacity-um -> unix gruma -> O(1)
```

Capacity lcvac — O(n) (reallocation)

```
size=4, capacity=4: [1][2][3][4]  
push_back(5):  
  1. nor memory allocate  
  2. capacity mecacnum (x2 -> 8)  
  3. bolor arjeqner copy/move  
  4. hin memory azatum
```

Ayd mek gorcoxutyuny O(n) e.

Chisht patasxany — Amortized O(1)

Vector-y capacity-n **krknapatkum** e (1,2,4,8,16,32,...), nuunh reallocation-nery **hazvadeq** en. Shat push_back-i mijin arjeqy darnum e **amortized O(1)**.

```
for (int i = 0; i < 1000000; i++)  
    v.push_back(i); // voroshy O(n), bayc mijiny push_back-i hamar O(1)
```

reserve()-ov — grethe voch mi reallocation

```
vector<int> v;  
v.reserve(1000000); // naxapes memory  
for (int i = 0; i < 1000000; i++)  
    v.push_back(i); // grethe voch mi reallocation -> arag
```

12. Interview հարցեր & patasxanner

Iterator inch e?

Object, vor tuyl e talis anc kacnel container-i element-nerov; pointer-i nman e ashxatum, talis miasnakan interface STL container-neri hamar.

begin() / end()?

begin() = araji element; end() = VERJINIC HETO diry (n₂ verjin element).

end()-y verjin element-n e?

*Voch. Verjinic heto e; *end() -> UB.*

Vector-i iterator inch tesak e?

Random Access Iterator.

List-i iterator inch tesak e?

Bidirectional Iterator.

Inchu list-um it+5 chi ashxatum?

list-y Bidirectional e (miayn ++/--), Random Access chuni.

push_back-ic heto iterator karo e invalid darnal?

Ayo, ete reallocation texi unena (vector).

List-um push_back-ic heto iterator invalid e?

Sovorabar voch (list stable e).

erase() inch e veradarcnum?

Hajord valid iterator-y.

const_iterator?

*Iterator, voricov kareli e kardal bayc voch poxel (*it = x -> error).*

++it vs it++?

*++it pre-increment (arag, reference); it++ post-increment (temporary object, mi qich tañk).
Naxentreli ` ++it.*

push_back complexity?

Amortized $O(1)$; reallocation-i depqum mek gorcoxutyun $O(n)$ (bolor element copy/move).

Cheat Sheet

ITERATOR = STL-i «xelaci pointer», container-i element-nerov anc

begin() = araji | end() = VERJINIC HETO (ne verjin!)

*it dereference | ++it araj | --it het (bidirectional)

TYPES (tuyl -> hzor):

Input/Output -> Forward -> Bidirectional (list,set,map) -> Random Access (vector,deque)

it+5 miayn Random Access-um (vector/deque)

INVALIDATION:

vector: push_back/insert/erase/resize/reserve -> realloc -> INVALID

list: insert/push -> STABLE (miayn erase-y jnjvac node)

ERASE IDIOM:

it = v.erase(it); // erase veradarcnum hajord iterator; ++it mi ara chuxtin

HELPERS: advance(it,n) [void] | next(it,n) | prev(it,n) | rbegin/rend | cbegin/cend

++it vs it++: naxentreli ++it (it++ -> temporary object)

push_back: amortized $O(1)$ | reallocation -> $O(n)$ | reserve() -> xusapum

pointer vs iterator: pointer=address | iterator=container-aware object